



INSTITUTO POLITECNICO NACIONAL

Escuela Superior de Computo

COMPILADORES

Práctica 3 – Análisis Descendiente Recursivo

Profesora: ALBORTANTE MORATO CECILIA

Alumnos: Gutiérrez Espinosa Jordan Gabriel
Ricardo Bisogno Gandarilla

Grupo: 5CM2

Ciclo: 2022-2023(2)



Introducción	1
---------------------	----------

DESARROLLO	2
-------------------	----------

Resultados	8
-------------------	----------

Referencias	9
--------------------	----------

INTRODUCCION

El análisis descendente recursivo es un método de análisis sintáctico utilizado en la programación de computadoras para determinar si una cadena de símbolos es parte de un lenguaje formal o no. Es un proceso de descomposición de una cadena en sus componentes más pequeños, siguiendo las reglas de una gramática formal [1].

En general, el análisis descendente recursivo es una técnica top-down, lo que significa que comienza por la raíz de un árbol sintáctico y desciende por sus ramas hasta llegar a las hojas, que son las terminales de la gramática. El proceso de análisis se realiza mediante un conjunto de funciones recursivas que corresponden a las reglas de la gramática. Cada función toma como entrada una parte de la cadena de entrada y devuelve un valor booleano que indica si la entrada coincide con la regla como se muestra en la Figura 1 [2].

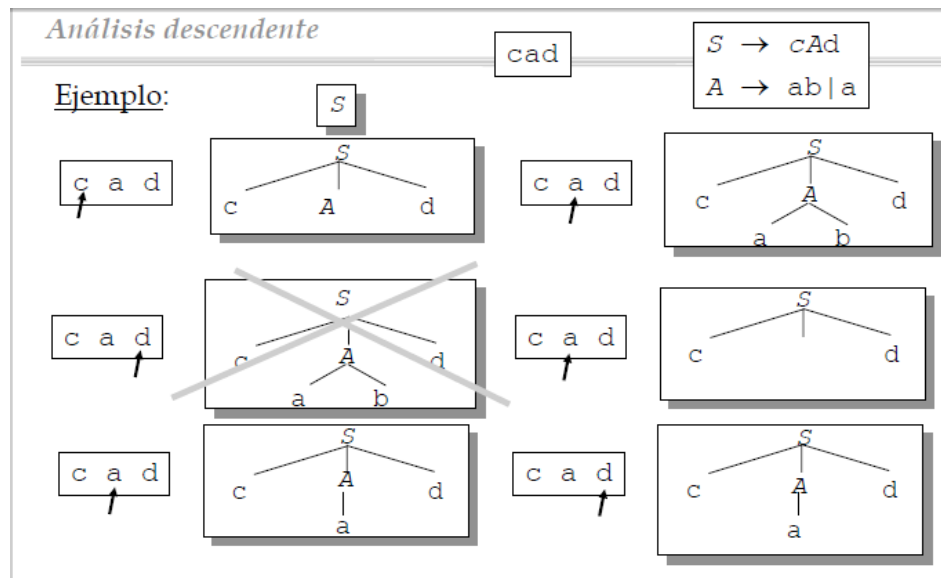


Figura 1, Analisador decediente

El análisis descendente recursivo se usa comúnmente en la construcción de compiladores para verificar la sintaxis de los programas escritos en un lenguaje de programación. En general, el proceso implica la construcción de un árbol sintáctico que representa la estructura de un programa y la verificación de que la entrada coincide con la gramática del lenguaje [3].

DESARROLLO

Main.java

En este código se recibe la cadena de tokens, su eliminación de espacios y la envía al analizador para su revisión y dar como resultado si es aceptado o no.

Como se muestra en la Figura 2 método "main" contiene un bucle "while" que se ejecuta continuamente mientras el usuario desee analizar más cadenas, Se crea un objeto "*Analizador sintactico*" que es la clase que se encarga de analizar la cadena, ya dentro del bucle se imprime un mensaje de bienvenida. Luego, se solicita al usuario que ingrese una cadena a analizar utilizando la función "*readLine()*" del objeto "*System.console()*". La cadena ingresada por el usuario se limpia de cualquier espacio en blanco utilizando un bucle "for" y luego se asigna al objeto "text" de la clase "*Analizador sintactico*".

```
public class Main {  
    Run | Debug  
    public static void main(String[] args) {  
        Analizador_sintactico text = new Analizador_sintactico();  
        System.out.println(x:"\n\t Practica 3: Analizador Sintactico Recursivo\n");  
        String preliminarText;  
        String finalText;  
        String resp;  
        while (true) {  
            text.setIndex(index:0);  
            System.out.print(s:"\n\t Por favor ingrese la cadena a analizar: ");  
            preliminarText = System.console().readLine();  
            finalText = "";  
            for (int i = 0; i < preliminarText.length(); i++) {  
                if (preliminarText.charAt(i) == ' ') {  
                    continue;  
                }  
                finalText += preliminarText.charAt(i);  
            }  
            text.setText(finalText);  
        }  
    }  
}
```

Figura 2, Clase main Lectura de la cadena.

A continuación como se muestra en la Figura 3, se llama al método "*analizar()*" del objeto "text" para realizar el análisis sintáctico de la cadena ingresada. Si el análisis es exitoso, es decir, la cadena cumple con las reglas sintácticas definidas en la clase "*Analizador sintactico*", se imprime un mensaje indicando que la palabra es reconocida. De lo contrario, se imprime un mensaje indicando que la palabra no es reconocida.

Por último, se le pregunta al usuario si desea analizar otra cadena y se lee su respuesta utilizando la función "*readLine()*" del objeto "*System.console()*". Si el usuario ingresa "n" o "N", el bucle "while" se interrumpe y la aplicación se detiene. De lo contrario, el bucle continúa y se solicita al usuario que ingrese otra cadena.

```
System.out.print(s:"\n");
if (text.analizar() == 1) {
    System.out.println(x:"\n La palabra es reconocida.");
} else {
    System.out.println(x:"\n La palabra no es reconocida.");
}
System.out.print(s:"\n Desea comprobar otra cadena? En caso de no digite n, en caso de si pres
resp = System.console().readLine();
if (resp.equals(anObject:"n") || resp.equals(anObject:"N")) {
    break;
}
```

Figura 3, Reconocimiento de la palabra y Bucle de peticiones.

Analizador_sintactico.java

Este código es una implementación de un analizador sintáctico básico en Java. El analizador toma la expresión de tokens matemáticos como entrada y verifica si la sintaxis de la expresión es correcta. El analizador utiliza una serie de funciones que representan la estructura de la expresión, y estas funciones se llaman de manera recursiva para analizar la expresión.

Como se muestra en la Figura 4, El código comienza importando la clase "*java.util.regex.**" para utilizar expresiones regulares en la identificación de patrones en la cadena de entrada. La clase "*Analizador_sintactico*" define las variables de instancia "text" e "index" y métodos para establecer y obtener los valores de estas variables.

```
import java.util.regex.*;

public class Analizador_sintactico {
    public String text;
    public int index;

    public Analizador_sintactico() {
        index = 0;
    }

    public void setIndex(int index) {
        this.index = index;
    }

    public void setText(String text) {
        this.text = text;
    }

    public int getIndex() {
        return index;
    }

    public String getText() {
        return text;
    }
}
```

Figura 4, Librería e identificadores.

La función "*analizar*" como se muestra en la Figura 5 es la función principal del analizador sintáctico, y se encarga de llamar a la función "*e*" que analiza los términos, operadores y paréntesis de la expresión. Si la función "*e*" devuelve un valor 0, significa que se encontró un error en la expresión; si devuelve un valor 1, la expresión es válida y si devuelve un valor 1 o 0, significa que se ha completado el análisis de la expresión.

```
public int analizar() {
    int comp, comp2 = 0, comp3 = 0;
    do {
        comp = e();
        if (comp == 3) {
            if (index >= text.length()) {
                comp3 = 1;
            } else if (text.charAt(index) == '(') {
                index++;
                comp2 = 0;
                do {
                    if (index >= text.length()) {
                        return comp;
                    }
                    comp = e();
                    if (comp == 0) {
                        return 0;
                    }
                    if (index == text.length()) {
                        return 0;
                    }
                    if (text.charAt(index) == ')') {
                        if (text.length() == (index + 1)) {
                            return 1;
                        } else {
                            index++;
                            comp2 = 1;
                        }
                    } else if (text.charAt(index) == '(') {
                        index++;
                        comp = analizar();
                    }
                } while (comp2 == 0);
            } else if (text.charAt(index) == ')') {
                return 3;
            }
        } else {
            comp3 = 1;
        }
    } while (comp3 == 0);
    return comp;
}
```

Figura 5, Funcion analizar.

La función "e" analiza los términos y operadores de la expresión llamando a la función "t", y verifica si hay un signo "+" o "-" después de un término. Si encuentra un signo "+" o "-", llama a la función "t" para analizar el siguiente término en la expresión. Si la función "t" devuelve un valor 0, significa que se encontró un error en la expresión; si devuelve un valor 1, la expresión es válida y si devuelve un valor 3, significa que el programa encontró un paréntesis, cuando encuentra un paréntesis se regresa hasta el método analizar y entra en un bucle diferente al anterior ya que adentro del paréntesis puede haber cualquier cosa, y hasta que no se cierre ese paréntesis no se sale de ese bucle, si no encontró ningún paréntesis de cierre y terminó la cadena regresa 0 como un error, Como se muestra en la Figura 6.

```
public int e() {
    int comp, comp2;
    do {
        comp = t();
        if (comp == 1) {
            if (index >= text.length()) {
                comp2 = 1;
            } else if (text.charAt(index) == '+' || text.charAt(index) == '-') {
                System.out.println(" Token: " + 2 + " id: " + text.charAt(index));
                index++;
                comp2 = 0;
            } else {
                comp2 = 1;
            }
        } else {
            comp2 = 1;
        }
    } while (comp2 == 0);

    if (comp == 0) {
        return 0;
    } else if (comp == 3) {
        return 3;
    } else {
        return 1;
    }
}
```

Figura 6, Función e.

La función "t" analiza los factores de la expresión llamando a la función "f", y verifica si hay un signo "*" o "/" después de un factor. Si encuentra un signo "*" o "/", llama a la función "f" para analizar el siguiente factor en la expresión. Si la función "f" devuelve un valor 0, significa que se encontró un error en la expresión; si devuelve un valor 1, la expresión es válida y si devuelve un valor 3, significa como en la Función "e" que el programa encontró un paréntesis, cuando encuentra un paréntesis se regresa hasta el método analizar y entra en un bucle diferente al anterior ya que adentro del paréntesis puede haber cualquier cosa, y hasta que no se cierre ese paréntesis no se sale de ese bucle, si no encontró ningún paréntesis de cierre y terminó la cadena regresa 0 como un error, Como se muestra en la Figura 7.

```
public int t() {
    int comp, comp2;
    do {
        comp = f();
        if (comp == 1 || comp == 2) {
            if (index >= text.length()) {
                comp2 = 1;
            } else if (text.charAt(index) == '/' || text.charAt(index) == '*') {
                System.out.println(" Token: " + 2 + " id: " + text.charAt(index));
                index++;
                comp2 = 0;
            } else if (text.charAt(index) == '(' || text.charAt(index) == ')') {
                comp2 = 0;
            } else {
                comp2 = 1;
            }
        } else {
            comp2 = 1;
        }
    } while (comp2 == 0);

    if (comp == 0) {
        return 0;
    } else if (comp == 3) {
        return 3;
    } else {
        return 1;
    }
}
```

Figura 7, Función t.

La función "f" analiza los factores de la expresión y devuelve un valor 1 si encuentra una variable o un valor numérico, entra en un bucle "do-while" para agrupar lo diferentes caracteres que se puedan encontrar, los clasifica gracias a la librería "regex", la manera en que funciona es que mientras encuentre una letra o un dígito va a continuar aumentando el "index" para agrupar las variables o enteros, sin embargo cuando se encuentra un signo de operación se rompe el bucle y se procesa la variable, al retornar una variable, se comprueba si hay un signo de operación o un paréntesis, Como se muestra en la Figura 8.

```
public int f() {
    int inicio = index;
    do {
        if ((index + 1) >= text.length()) {
            if (index > 0) {
                if (text.charAt(index - 1) == '(' && text.charAt(index) == ')') {
                    System.out.println(" Token: " + 2 + " id: " + text.substring(inicio, index + 1));
                    return 0;
                } else if (text.charAt(index - 1) == ')' && text.charAt(index) != ')') {
                    return 2;
                }
            }
            if (text.charAt(index) == '(' || text.charAt(index) == ')') {
                break;
            }
            index++;
            break;
        } else {
            if (index > 0) {
                if (text.charAt(index - 1) == '(' && text.charAt(index) == ')') {
                    System.out.println(" Token: " + 2 + " id: " + text.substring(inicio, index + 1));
                    return 0;
                } else if (text.charAt(index - 1) == ')' && text.charAt(index) != ')') {
                    return 2;
                }
            }
            if (text.charAt(index) == '(' || text.charAt(index) == ')') {
                break;
            }
            index++;
        }
    }
}
```

Figura 8, Función f.

Por último, el código utiliza "*System.out.println*" para imprimir los tokens de la expresión y sus identificadores durante el proceso de análisis, ya en la Clase "*main*" se recibió el proceso del análisis indicando al usuario si fue aceptado o no la cadena, Como se muestra en la Figura 9.

```
} while (Pattern.matches(regex:"[\\w]", text.charAt(index) + "") && index < text.length());
if (Pattern.matches(regex:"[\\w]+", text.substring(inicio, index))) {
    System.out.println(" Token: " + 1 + " id: " + text.substring(inicio, index));
    return 1;
} else if (Pattern.matches(regex:"[\\d]+", text.substring(inicio, index))) {
    System.out.println(" Token: " + 3 + " id: " + text.substring(inicio, index));
    return 2;
} else if (text.charAt(index) == '(' || text.charAt(index) == ')') {
    System.out.println(" Token: " + 2 + " id: " + text.substring(inicio, index + 1));
    return 3;
} else {
    return 0;
}
```

Figura 9, Escritura de los tokens.

Resultados

En la Figura 10, se muestra el título de la practica y la solicitud al usuario de colocar la cadena a analizar, en este caso la cadena “3+2”.

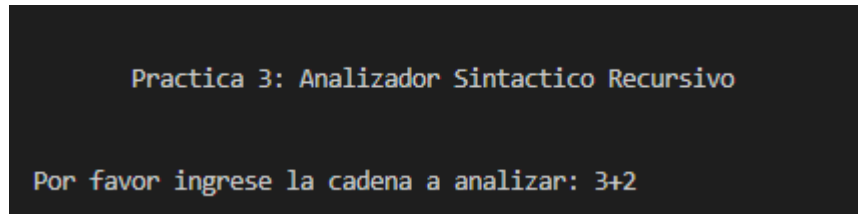


Figura 10, Solicitud de la cadena.

En la figura 11, se muestra como el programa coloca parte por parte los tokens y su identificador y si la palabra es aceptada.

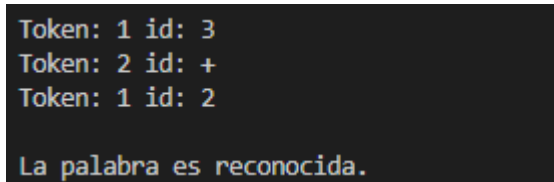


Figura 11, Identificación de los Tokens y la palabra.

Al final se pide si desea introducir otra palabra, y como se muestra en la Figura 12, se coloca otra palabra para ver si la reconoce en este caso la cadena “33+2-a” la cual tiene un identificador, reconociéndola.

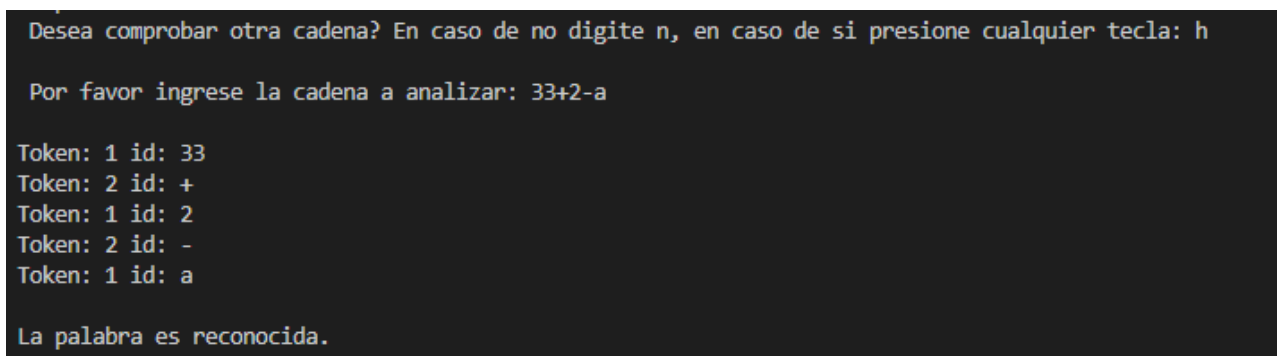


Figura 12, Bucle y reconocimiento de cadena con identificador.



Referencias

- [1] M. A. Hicks, "An introduction to parsing: Top-down and bottom-up," IEEE Potentials, vol. 23, no. 4, pp. 6-10, Jul./Aug. 2004, <https://ricardogeek.com/tipos-de-parsers-en-un-compilador/>.
- [2] S. Rajkumar and S. Chakraborty, "A survey of parsing techniques," in 2012 IEEE International Conference on Advanced Communication Control and Computing Technologies (ICACCCT), pp. 1229-1234, Jun. 2012, <https://www.cs.ox.ac.uk/files/219/parsing.pdf>
- [3] A. V. Aho, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Reading, MA, USA: Addison-Wesley Publishing Company, Inc., 2006 <https://www.dbscience.org/wp-content/uploads/2020/03/ALSUdragonbookcompilers.pdf>.